Hortonworks Data Platform 2.6

Sandbox on VirtualBox

Lab Guide — Introductory Big Data Course

HDFS Distributed Storage	**MapReduce** Batch Processing	**Hive** SQL-style Queries	**HBase** NoSQL Store

1. Introduction

1.1 What is Apache Hadoop?

Apache Hadoop is an open-source framework that enables the distributed processing of very large datasets across clusters of commodity hardware. Originally inspired by Google's internal infrastructure papers (GFS and MapReduce), it was developed at Yahoo! and released as an Apache project in 2006. Today it sits at the core of what the industry calls the Big Data ecosystem.

Hadoop addresses a fundamental challenge in modern computing: datasets have grown so large — from gigabytes to terabytes and petabytes — that a single machine can no longer store or process them within a reasonable time. Hadoop's answer is to split both the data and the computation across many machines, coordinating their work transparently.

The Hadoop ecosystem is built on two foundational components:

- **HDFS (Hadoop Distributed File System):** the storage layer, which breaks files into blocks and replicates them across the cluster for fault tolerance.
- **YARN (Yet Another Resource Negotiator):** the resource management and job scheduling layer, which allocates CPU and memory across running applications.

Over time, a rich set of tools has been built on top of these two foundations — Hive for SQL-style analytics, HBase for low-latency NoSQL access, Pig for scripting, Spark for in-memory processing, Zookeeper for coordination, and many others.

1.2 What is Hortonworks Data Platform (HDP)?

Hortonworks Data Platform (HDP) is an enterprise distribution of the Apache Hadoop ecosystem, developed by Hortonworks (now part of Cloudera). HDP packages together a curated and tested set of Hadoop components, adds management tooling (Apache Ambari), and provides a stable, supported environment for production use.

Version 2.6, released in 2017, is a mature and widely documented release that includes:

- Apache Hadoop 2.7.x (HDFS + YARN)
- Apache Hive 1.2 / 2.1 (SQL-on-Hadoop)
- Apache HBase 1.1 (NoSQL column store)
- Apache Pig 0.16 (data flow scripting)
- Apache Spark 2.1 (in-memory processing)
- Apache Zeppelin 0.7 (interactive notebooks)
- Apache Kafka 0.10 (streaming)
- Apache Ambari 2.5 (cluster management UI)

For teaching purposes, HDP 2.6 is an excellent choice: it is stable, well-documented, and all its components are pre-installed and pre-configured in the Sandbox VM, allowing students to focus on learning rather than on infrastructure setup.

1.3 The HDP Sandbox

The HDP Sandbox is a single-node virtual machine that simulates a Hadoop cluster on a personal laptop or desktop. It is distributed as an OVA (Open Virtual Appliance) file that can be imported directly into VirtualBox. Once running, it exposes all the standard Hadoop services as if you were connected to a full cluster.

Learning Note: In production, Hadoop runs on dozens or hundreds of physical machines. The Sandbox collapses all cluster roles — NameNode, DataNode, ResourceManager, NodeManager, HBase Master, Region Server — onto a single virtual machine. This is purely for learning; the programming model and interfaces are identical to a real cluster.

The Sandbox is the standard starting point for learning Hadoop and is used in official Hortonworks tutorials. Working through this lab guide, you will gain hands-on experience with the four core areas of the ecosystem: distributed storage, batch processing, SQL analytics, and NoSQL access.

2. Prerequisites & System Requirements

2.1 Hardware Requirements

The HDP Sandbox is a resource-intensive virtual machine. Before proceeding, verify that your machine meets the following requirements:

Component	Minimum	Recommended
RAM	8 GB	12–16 GB
Disk Space	50 GB free	80 GB free
CPU Cores	2 cores	4 cores
VirtualBox	Version 5.1+	Version 6.x
OS	Windows 7 / macOS 10.12 / Ubuntu 16.04	Windows 10 / macOS 10.14+

Important: The most critical factor is RAM. Allocating less than 8 GB to the VM will cause services to fail during startup. If your machine has exactly 8 GB of physical RAM, close all other applications before starting the VM.

2.2 Required Software

Before importing the Sandbox, install the following software on your host machine:

- **Oracle VirtualBox 5.1 or later** — available at www.virtualbox.org. Version 6.x is strongly recommended for better performance and compatibility.
- **VirtualBox Extension Pack** — must match your VirtualBox version. Download from the same page. Provides USB 2.0/3.0 support and improved networking.
- **An SSH client** — on Windows, use PuTTY or Windows Terminal. On macOS/Linux, the built-in terminal is sufficient.

2.3 Downloading the HDP Sandbox

The HDP 2.6 Sandbox OVA file can be obtained from the Hortonworks/Cloudera documentation archive. Search for "HDP 2.6 Sandbox VirtualBox" to locate the official download. The file is approximately 12–14 GB, so ensure you have a stable internet connection and sufficient disk space before downloading.

Note: The file extension will be .ova (Open Virtual Appliance). Do not unzip or modify this file — VirtualBox imports it directly.

3. VirtualBox Setup & Import

3.1 Importing the OVA File

Follow these steps to import the Sandbox into VirtualBox:

1. Open VirtualBox and go to File → Import Appliance (or press Ctrl+I / Cmd+I).
2. Click the folder icon and navigate to the downloaded .ova file. Select it and click Next.
3. Review the Appliance Settings screen. You will see a list of virtual hardware configurations.
4. Adjust the RAM allocation: find the Memory row and set it to at least 8192 MB (8 GB). If your machine has 16 GB or more, set it to 10240–12288 MB for better performance.
5. Leave all other settings at their defaults. Do not change the number of CPUs below 2.
6. Click Import. The process will take 5–15 minutes depending on your machine's storage speed.

Warning: Do not interrupt the import process. If it fails partway through, delete the partial VM from the VirtualBox Machine list and restart the import from scratch.

3.2 Network Configuration

Correct network configuration is essential for accessing the Sandbox services from your host browser and terminal. The recommended setup uses Host-Only Adapter with NAT.

Option A: NAT + Port Forwarding (Recommended for most users)

This is the default configuration after import. VirtualBox automatically maps the VM's internal ports to your localhost. With this setup, all services are accessible at localhost (127.0.0.1) on the forwarded ports. No additional configuration is needed.

Option B: Host-Only Adapter

If you need direct network access to the VM (for multi-node simulations or IDE integration), right-click the VM → Settings → Network → Adapter 1 → Attached to: Host-Only Adapter. Note the IP address shown on the VM console after boot and use it instead of localhost.

3.3 Verifying Port Forwarding Rules

To check or modify the port forwarding rules (NAT mode):

7. Right-click the VM in VirtualBox → Settings → Network.

8. Expand Adapter 1 (should show NAT) and click Advanced → Port Forwarding.
9. Verify the following rules are present (add any that are missing):

Service	Port	Access URL / Command
Ambari Dashboard	8080	http://localhost:8080
SSH (Terminal)	2222	ssh root@localhost -p 2222
HDFS NameNode UI	50070	http://localhost:50070
YARN ResourceManager	8088	http://localhost:8088
HBase Master UI	16010	http://localhost:16010
Zeppelin Notebook	9995	http://localhost:9995
Hive Server2	10000	JDBC / Beeline CLI

4. First Boot & Access

4.1 Starting the Virtual Machine

Select the HDP Sandbox VM in VirtualBox and click Start (or double-click it). A console window will appear showing the Linux boot sequence.

Patience Required: The first boot takes significantly longer than subsequent boots — typically 3 to 8 minutes. Hadoop services start sequentially and each one must initialize before the next can begin. Do not close the console window or shut down the VM during this process.

The VM is ready when the console displays a screen similar to the following, showing the Sandbox welcome message and the access URLs:

```
Hortonworks Sandbox

   hostname: sandbox.hortonworks.com
   Ambari: http://localhost:8080

   To get started, log into Ambari.
   username: admin   password: admin

   SSH:  ssh root@localhost -p 2222
         default password: hadoop
```

4.2 Accessing the Sandbox

There are three primary ways to interact with the Sandbox. All three are used in the exercises in this guide.

Method 1: Ambari Web Interface

Ambari is the cluster management dashboard. Open your browser and navigate to:

```
http://localhost:8080
```

Log in with username **admin** and password **admin**. The Ambari dashboard gives you a graphical overview of all running services, their health status, and configuration options.

Method 2: SSH Terminal

For command-line access, open a terminal and connect via SSH:

```
ssh root@localhost -p 2222
# Default password: hadoop
```

On Windows, configure PuTTY with Host: localhost, Port: 2222, Connection type: SSH.

Method 3: Ambari Shell (Browser-Based)

If you cannot use SSH, Ambari provides a web-based shell. After logging into Ambari, click the grid icon in the top right, select Shell Access, and a terminal will open inside your browser.

4.3 Checking Service Health

After the VM finishes booting, log into Ambari and check the Services panel on the left. Each service should show a green icon. If any service shows red or orange:

- Click the service name to open its detail page.
- Click Service Actions → Start to restart it.
- Wait 60–90 seconds and check again.

The services we will use in this lab are: HDFS, YARN, MapReduce2, Hive, HBase, and Zeppelin. Ensure all of these are running (green) before proceeding to the exercises.

5. Environment Overview

5.1 The Hadoop Ecosystem Architecture

Before diving into exercises, it is important to understand how the components relate to each other. Think of the Hadoop ecosystem as a stack of layers:

Interactive Layer: Zeppelin Notebooks
Query Layer: Hive (SQL) | HBase (NoSQL) | Pig (Scripting)
Processing Layer: MapReduce | Apache Spark
Resource Management: YARN
Storage Layer: HDFS (Hadoop Distributed File System)

Data flows upward through these layers: it is stored in HDFS, resources are managed by YARN, computations happen via MapReduce or Spark, and higher-level tools like Hive and HBase provide convenient interfaces for specific use cases.

5.2 Key Concepts

HDFS — Distributed File System

HDFS divides files into blocks (typically 128 MB each) and distributes them across DataNodes. Each block is replicated (default: 3 copies) across different nodes for fault tolerance. The NameNode maintains the file system metadata (directory tree, block locations), while DataNodes store the actual data.

YARN — Resource Management

YARN separates resource management from processing logic. The ResourceManager allocates cluster resources to applications. Each application has an ApplicationMaster that negotiates containers (bundles of CPU + RAM) with the ResourceManager and coordinates its tasks.

MapReduce — Batch Processing

MapReduce is a programming model for parallel batch processing. A job is divided into two phases: the Map phase transforms input records into key-value pairs, and the Reduce phase aggregates all values for each key. YARN orchestrates the execution across the cluster.

Hive — SQL on Hadoop

Hive provides a SQL-like query language (HiveQL) that translates to MapReduce or Tez jobs running on YARN. It is ideal for analysts familiar with SQL who need to query large datasets stored in HDFS without writing Java code.

HBase — NoSQL Column Store

HBase is a distributed, sparse, persistent multidimensional sorted map built on top of HDFS. It organizes data into tables with rows and column families. Unlike Hive, HBase supports random, low-latency read/write access — making it suitable for operational use cases where millisecond response time is needed.

Apache Ambari — Cluster Management

Ambari is the web-based management tool for HDP clusters. It monitors service health, allows configuration changes, displays logs, and provides an alert system. In the Sandbox, it is your primary dashboard for understanding the state of the cluster.

6. Practical Exercises

The following exercises provide hands-on experience with the four core components of the HDP ecosystem. Each exercise is self-contained and explains both the commands and the underlying concept.

Before starting: Connect to the Sandbox via SSH: ssh root@localhost -p 2222 (password: hadoop). All command-line exercises in this section should be run from within this SSH session.

Exercise 1: HDFS — Distributed File Storage

Objective: Learn the basic HDFS operations — creating directories, uploading files, listing contents, and retrieving files.

Step 1.1 — Explore the HDFS Root

HDFS has its own file system hierarchy, separate from the Linux filesystem. Use the hdfs dfs command to interact with it:

```
# List the HDFS root directory
hdfs dfs -ls /

# List the /user directory
hdfs dfs -ls /user

# Show disk usage of HDFS
hdfs dfs -df -h /
```

What you will see: The /user directory contains home directories for each Hadoop user. The /tmp directory is used by various services. The /apps directory contains Hive and other application data.

Step 1.2 — Create a Directory and Upload a File

```
# Create a sample text file on the local Linux filesystem
echo 'Hello from HDFS!' > /tmp/sample.txt
echo 'Big Data is about storing and processing' >> /tmp/sample.txt
echo 'very large volumes of information.' >> /tmp/sample.txt
```

```
# Create a directory in HDFS
hdfs dfs -mkdir -p /user/root/lab01

# Upload the file to HDFS
hdfs dfs -put /tmp/sample.txt /user/root/lab01/

# Verify the upload
hdfs dfs -ls /user/root/lab01/
```

Step 1.3 — Read and Download Files

```
# Read the file directly from HDFS (like 'cat')
hdfs dfs -cat /user/root/lab01/sample.txt

# Download the file back to the local filesystem
hdfs dfs -get /user/root/lab01/sample.txt /tmp/sample_downloaded.txt

# Verify the downloaded file
cat /tmp/sample_downloaded.txt

# Display HDFS block and replication information
hdfs fsck /user/root/lab01/sample.txt -files -blocks
```

What is happening: When you run hdfs fsck you will see block information showing that even a small file is stored with 3 replicas. In a real cluster these 3 copies would live on 3 different physical machines, protecting against hardware failure.

Step 1.4 — HDFS Web UI

Open your browser and go to http://localhost:50070. This is the HDFS NameNode UI. Explore the following sections:

- Overview: shows cluster capacity, used space, and node count.
- Datanodes: lists all DataNodes (in the Sandbox there is only one).
- Browse the filesystem: navigate the HDFS directory tree visually.

Exercise 2: MapReduce — Word Count

Objective: Run the classic MapReduce WordCount example to understand the Map-Shuffle-Reduce pipeline and observe job execution through the YARN UI.

Step 2.1 — Prepare the Input Data

```
# Create a larger text file for a more interesting word count
cat > /tmp/text_input.txt << 'EOF'
Apache Hadoop is an open source framework for distributed computing
Hadoop enables processing of large datasets across clusters of computers
The Hadoop Distributed File System stores data reliably across the cluster
MapReduce is a programming model for processing large data sets
Hadoop MapReduce is a software framework for easily writing applications
Apache Hadoop YARN is the resource management platform responsible for
managing computing resources in clusters and using them for scheduling
Hadoop has become the standard for big data processing in the industry
EOF

# Upload to HDFS
hdfs dfs -mkdir -p /user/root/wordcount/input
hdfs dfs -put /tmp/text_input.txt /user/root/wordcount/input/
hdfs dfs -ls /user/root/wordcount/input/
```

Step 2.2 — Run the WordCount MapReduce Job

```
# Run the WordCount example from the Hadoop examples JAR
yarn jar /usr/hdp/current/hadoop-mapreduce-client/hadoop-mapreduce-examples.jar \
  wordcount \
  /user/root/wordcount/input \
  /user/root/wordcount/output

# The job will print progress like:
# INFO mapreduce.Job: Running job: job_1234567890_0001
# INFO mapreduce.Job: map 0% reduce 0%
# INFO mapreduce.Job: map 100% reduce 0%
# INFO mapreduce.Job: map 100% reduce 100%
# INFO mapreduce.Job: Job job_1234567890_0001 completed successfully
```

Step 2.3 — View the Results

```
# List the output directory
hdfs dfs -ls /user/root/wordcount/output/

# View the word count results
hdfs dfs -cat /user/root/wordcount/output/part-r-00000

# Sort by count (highest first) — pipe through local sort
hdfs dfs -cat /user/root/wordcount/output/part-r-00000 | sort -k2 -rn | head -15
```

What is happening: The Map phase reads each line and emits (word, 1) pairs. The Shuffle phase groups all pairs by word. The Reduce phase sums all the 1s for each word. The output file part-r-00000 contains the final (word, count) pairs sorted alphabetically.

Step 2.4 — Monitor via YARN UI

Open http://localhost:8088 in your browser. This is the YARN ResourceManager UI. Click on the completed job to see:

- Map and Reduce task counts and their execution times.
- Container allocation (memory and CPU used).
- Job counters showing bytes read/written and records processed.

Exercise 3: Hive — SQL Analytics on HDFS

Objective: Use HiveQL to create tables backed by HDFS data, load records, and run analytical SQL queries — demonstrating how Hive bridges the SQL world with the Hadoop ecosystem.

Step 3.1 — Prepare Sample Data

We will use a small sales dataset to simulate a realistic analytics scenario.

```
# Create a CSV file with sales data
cat > /tmp/sales.csv << 'EOF'
1,Alice,Electronics,1200.00,2023-01-15
2,Bob,Clothing,89.50,2023-01-16
3,Alice,Books,45.00,2023-01-17
4,Carol,Electronics,799.99,2023-01-18
5,Bob,Electronics,299.00,2023-01-19
6,Carol,Books,120.00,2023-01-20
7,Alice,Clothing,210.00,2023-01-21
8,Bob,Books,35.00,2023-01-22
9,Carol,Clothing,450.00,2023-01-23
10,Alice,Electronics,599.00,2023-01-24
EOF

# Upload to HDFS
hdfs dfs -mkdir -p /user/root/hive_data/sales
hdfs dfs -put /tmp/sales.csv /user/root/hive_data/sales/
```

Step 3.2 — Open the Hive CLI

```
# Launch the Beeline Hive CLI
beeline -u jdbc:hive2://localhost:10000 -n root
```

Alternative: If Beeline is slow to connect, you can also use the older Hive CLI by typing: hive

Step 3.3 — Create and Populate a Table

```
-- Create a database for our lab
CREATE DATABASE IF NOT EXISTS lab_db;
USE lab_db;

-- Create an external table pointing to our HDFS data
CREATE EXTERNAL TABLE IF NOT EXISTS sales (
  sale_id     INT,
  customer    STRING,
  category    STRING,
  amount      DOUBLE,
  sale_date   STRING
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
STORED AS TEXTFILE
LOCATION '/user/root/hive_data/sales';

-- Verify the table was created
SHOW TABLES;
DESCRIBE sales;
```

Step 3.4 — Run Analytical Queries

```
-- Basic select: view all records
SELECT * FROM sales;

-- Total sales per category
SELECT category,
       COUNT(*) AS num_sales,
       ROUND(SUM(amount), 2) AS total_revenue
FROM sales
GROUP BY category
ORDER BY total_revenue DESC;

-- Top customer by revenue
SELECT customer,
       ROUND(SUM(amount), 2) AS total_spent
FROM sales
GROUP BY customer
ORDER BY total_spent DESC
```

```
LIMIT 1;

-- Sales in the Electronics category above $500
SELECT customer, amount, sale_date
FROM sales
WHERE category = 'Electronics'
  AND amount > 500.00
ORDER BY amount DESC;
```

What is happening: Each HiveQL query is translated into one or more MapReduce or Tez jobs that run on YARN. You will see job progress printed in the console. The results are computed over data stored in HDFS — no data has been moved to a traditional database.

```
-- Exit Beeline when done
!quit
```

Exercise 4: HBase — NoSQL Column-Family Store

Objective: Experience the NoSQL paradigm through HBase — create a table with column families, insert records, perform key-based lookups, and understand why this model is fundamentally different from both SQL and HDFS batch access.

Why HBase? — The Motivation

Consider this problem: you have a website with 50 million users and you need to retrieve a specific user's profile in under 10 milliseconds. Hive (MapReduce) would scan the entire dataset — taking minutes. A traditional SQL database would struggle at this scale. HBase is designed exactly for this scenario: random, low-latency access to individual rows within massive tables.

Aspect	Hive (SQL)	HBase (NoSQL)
Access pattern	Full table scans / analytics	Random row lookups
Latency	Minutes (batch)	Milliseconds
Data model	Rows and columns (rigid schema)	Row key + column families (flexible)
Best for	Aggregation, reporting, BI	Operational apps, real-time access

Query language	HiveQL (SQL-like)	HBase Shell / Java API

Step 4.1 — The HBase Data Model

HBase organizes data differently from relational databases. Key concepts:

- **Row Key:** the unique identifier for each row (like a primary key), used for fast lookup.
- **Column Family:** a named group of related columns. Defined at table creation and stored together on disk.
- **Column Qualifier:** a specific column within a column family. Can be created dynamically — no fixed schema.
- **Cell:** the intersection of a row key and a column. Each cell can store multiple timestamped versions.

A column is always referenced as: family:qualifier (e.g., personal_info:name or activity:last_login).

Step 4.2 — Launch the HBase Shell

```
# Start the HBase interactive shell
hbase shell
```

Step 4.3 — Create a Table

We will model a user profile system — a classic HBase use case. Each user is a row, with two column families: one for personal information and one for account activity.

```
# Create the users table with two column families
create 'users', 'personal_info', 'activity'

# Verify the table was created
list

# Describe the table structure
describe 'users'
```

Step 4.4 — Insert Records with put

```
# Insert user 'alice' — personal information
put 'users', 'alice', 'personal_info:name', 'Alice Ferreira'
put 'users', 'alice', 'personal_info:email', 'alice@example.com'
put 'users', 'alice', 'personal_info:city', 'Sao Paulo'

# Insert user 'alice' — activity data
```

```
put 'users', 'alice', 'activity:last_login', '2024-03-10'
put 'users', 'alice', 'activity:logins_count', '142'
put 'users', 'alice', 'activity:plan', 'premium'

# Insert user 'bob'
put 'users', 'bob', 'personal_info:name', 'Bob Santos'
put 'users', 'bob', 'personal_info:email', 'bob@example.com'
put 'users', 'bob', 'personal_info:city', 'Rio de Janeiro'
put 'users', 'bob', 'activity:last_login', '2024-03-08'
put 'users', 'bob', 'activity:logins_count', '57'
put 'users', 'bob', 'activity:plan', 'free'

# Insert user 'carol'
put 'users', 'carol', 'personal_info:name', 'Carol Lima'
put 'users', 'carol', 'personal_info:email', 'carol@example.com'
put 'users', 'carol', 'personal_info:city', 'Brasilia'
put 'users', 'carol', 'activity:last_login', '2024-03-12'
put 'users', 'carol', 'activity:logins_count', '318'
put 'users', 'carol', 'activity:plan', 'enterprise'
```

Step 4.5 — Query with get and scan

```
# Get a single row by its row key (fast, key-based lookup)
get 'users', 'alice'

# Get only the personal_info column family for alice
get 'users', 'alice', 'personal_info'

# Get a specific cell
get 'users', 'alice', 'activity:plan'

# Scan all rows in the table
scan 'users'

# Scan only the activity column family
scan 'users', {COLUMNS => ['activity']}

# Count total rows in the table
count 'users'
```

Step 4.6 — HBase Versioning

One powerful HBase feature is that each cell can store multiple versions of a value, timestamped automatically. Let's observe this:

```
# Update alice's plan
put 'users', 'alice', 'activity:plan', 'enterprise'

# Get current value
```

```
get 'users', 'alice', 'activity:plan'

# Get with version history (up to 3 versions)
get 'users', 'alice', {COLUMN => 'activity:plan', VERSIONS => 3}

# You will see both 'premium' and 'enterprise' entries
# each with a different timestamp (Unix milliseconds)
```

What is happening: HBase stores each update as a new cell version tagged with a timestamp. By default only the latest version is returned, but you can access the full history. This is extremely useful for audit trails, event logging, and time-series data.

Step 4.7 — Cleanup

```
# Disable and delete the table when done
disable 'users'
drop 'users'

# Exit the HBase shell
exit
```

Exercise 5: Zeppelin — Interactive Notebook

Objective: Use Apache Zeppelin to run Hive queries interactively in a notebook interface, mixing code, results, and narrative documentation in a single document.

Step 5.1 — Access Zeppelin

Open your browser and go to http://localhost:9995. Log in with username admin and password admin.

Zeppelin is a web-based notebook similar to Jupyter. Each notebook contains paragraphs that can hold code in different interpreters (%hive, %spark, %sh, %md, etc.).

Step 5.2 — Create a New Notebook

10. Click Create new note.
11. Give it a name: Big Data Lab — Sales Analysis.

12. Leave the default interpreter as %hive.
13. Click Create.

Step 5.3 — Run Queries in Paragraphs

In the first paragraph, type and run (Shift+Enter or click the play button):

```
%hive
USE lab_db;
SELECT category,
       COUNT(*) AS num_sales,
       ROUND(SUM(amount), 2) AS total_revenue
FROM sales
GROUP BY category
ORDER BY total_revenue DESC;
```

Zeppelin will display the results as a table. You can switch to a bar chart by clicking the chart icon above the results.

Add a second paragraph with:

```
%hive
SELECT customer,
       ROUND(SUM(amount), 2) AS total_spent,
       COUNT(*) AS transactions
FROM lab_db.sales
GROUP BY customer
ORDER BY total_spent DESC;
```

What is happening: Zeppelin sends your HiveQL to the Hive interpreter, which runs it as a YARN job. The results come back and are rendered interactively in the browser. This is how data analysts typically interact with Hadoop in a modern workflow — without needing to use the command line.

7. Troubleshooting & Common Issues

7.1 VM Will Not Boot

- **Virtualization disabled in BIOS:** Enable Intel VT-x or AMD-V in your BIOS/UEFI settings. Look for Virtualization Technology or SVM Mode.
- **Hyper-V conflict (Windows):** If using Windows, Hyper-V must be disabled. Run in PowerShell as admin: bcdedit /set hypervisorlaunchtype off and reboot.
- **Not enough RAM:** Ensure you have at least 8 GB allocated. Reduce the VM RAM assignment to 8192 MB and close all other applications.

7.2 Services Show Red in Ambari

- **Services still starting:** Wait 3–5 more minutes. On first boot, services take time to initialize sequentially.
- **Manual restart:** Click the service name → Service Actions → Restart. If restarting fails, try Stop then Start.
- **Memory pressure:** If multiple services fail, the VM may be running out of memory. Increase the RAM allocation in VirtualBox settings (requires VM to be powered off).

7.3 Cannot Connect via SSH

- **Wrong port:** SSH is on port 2222, not the default 22. Use: ssh root@localhost -p 2222
- **Host key changed:** If you reinstalled the Sandbox, run: ssh-keygen -R [localhost]:2222 to clear the old key.
- **VM not fully booted:** Wait for the welcome screen on the VirtualBox console before attempting to connect.

7.4 Browser Cannot Reach localhost Ports

- **Port forwarding not set up:** Check VirtualBox Settings → Network → Port Forwarding and verify all ports from Section 3.3 are listed.
- **Service not running:** Confirm the service is green in Ambari before trying to access its UI.
- **Firewall blocking:** Temporarily disable your host firewall to test. If that resolves it, add exceptions for the required ports.

7.5 Hive Queries Hang or Time Out

- **YARN out of memory:** Reduce concurrent queries. Run one at a time.
- **HiveServer2 not running:** Check Hive service in Ambari and restart if needed.

- **Use Tez instead of MapReduce:** In the Beeline session, run: set hive.execution.engine=tez; for faster execution.

7.6 Saving and Restoring the VM State

To avoid waiting for a full boot each session, use VirtualBox snapshots:

- After first successful boot with all services running: Machine → Take Snapshot → name it "All Services Running".
- To safely shut down: in the Sandbox terminal, run sudo shutdown -h now, then close VirtualBox.
- To restore a previous state: Machine → Snapshots → select snapshot → Restore.

8. Summary & Next Steps

In this lab you have set up and explored the HDP 2.6 Sandbox, gaining practical experience with four foundational components of the Hadoop ecosystem:

HDFS	Distributed file storage with replication and fault tolerance
MapReduce	Parallel batch processing with automatic distribution across nodes
Hive	SQL-style analytical queries on HDFS data via MapReduce/Tez
HBase	NoSQL column-family store for low-latency random access at scale
Zeppelin	Interactive notebook interface for exploratory data analysis

Next Steps

To deepen your understanding of the ecosystem, explore the following topics in subsequent lab sessions:

- Apache Pig: write data transformation scripts using Pig Latin for ETL workflows.
- Apache Spark: run in-memory processing jobs that are 10–100x faster than MapReduce for iterative algorithms.
- Apache Kafka: build streaming data pipelines that ingest real-time events into HDFS or HBase.
- Apache Sqoop: import data from relational databases (MySQL, PostgreSQL) into HDFS for Hive analysis.
- Apache Oozie: orchestrate multi-step Hadoop workflows as scheduled or event-triggered jobs.

Further Reading: The official Hortonworks tutorials at docs.cloudera.com/HDPDocuments/HDP2/HDP-2.6.5/ provide step-by-step guides for each of these components. The Apache documentation for each project is also an excellent reference.